

LAPORAN PRAKTIKUM

STRUKTUR DATA

“IMPLEMENTASI SORTING DALAM BAHASA PEMROGRAMAN JAVA”

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi Sorting dalam Bahasa Pemrograman Java. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 20 Mei 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
1.3 Manfaat.....	3
BAB 2 PEMBAHASAN.....	5
2.1 Pengurutan menggunakan Insertion Sort.....	5
2.2.1 Method InsertionSort_2511531002().....	5
2.1.2 Method main().....	6
2.2 Pengurutan menggunakan Selection Sort.....	7
2.2.1 Method SelectionSort_2511531002().....	7
2.2.2 Method main().....	8
2.3 Pengurutan menggunakan Bubble Sort.....	9
2.3.1 Method BubbleSort_2511531002().....	9
2.3.2 Method main().....	10
2.4 Pengurutan menggunakan Insertion Sort menggunakna GUI.....	11
2.4.1 Deklarasi Class dan Variabel.....	11
2.4.2 Constructor InsertionSortGUI_2511531002().....	11
2.4.3 Method setArrayFromInput_1002().....	14
2.4.4 Method performStep_1002().....	16
2.4.5 Method updateLabels_1002().....	17
2.4.6 Method reset_1002().....	17
2.4.7 Method updateLabels_1002().....	18
2.4.8 Method main().....	19
2.5 Latihan Sorting dengan GUI untuk mengurutkan Nama Mahasiswa.....	20
BAB III KESIMPULAN.....	31
3.1 Kesimpulan.....	31
DAFTAR PUSTAKA.....	32

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sorting merupakan salah satu proses penting dalam struktur data yang digunakan untuk mengurutkan data berdasarkan aturan tertentu, seperti urutan ascending maupun descending. Proses pengurutan data sangat membantu dalam pengolahan informasi karena data yang tersusun rapi akan lebih mudah dicari, ditampilkan, dan dianalisis.

Dalam pemrograman, terdapat berbagai metode sorting yang memiliki cara kerja berbeda-beda, di antaranya Insertion Sort, Selection Sort, dan Bubble Sort. Ketiga metode tersebut sering digunakan sebagai dasar pembelajaran algoritma pengurutan karena konsepnya sederhana dan mudah dipahami.

Pada praktikum ini dilakukan implementasi beberapa algoritma sorting menggunakan bahasa pemrograman Java. Selain menampilkan hasil pengurutan dalam bentuk console, program juga dikembangkan menggunakan GUI (Graphical User Interface) agar proses sorting dapat divisualisasikan langkah demi langkah. Dengan adanya visualisasi tersebut, proses perpindahan dan pengurutan data menjadi lebih mudah dipahami.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar algoritma sorting.
2. Mengimplementasikan metode *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* menggunakan bahasa Java.
3. Memahami proses pengurutan data secara langkah demi langkah.
4. Membuat visualisasi proses sorting menggunakan GUI Java Swing.
5. Mengimplementasikan proses sorting pada data object mahasiswa menggunakan **ArrayList**.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menambah pemahaman mengenai konsep dasar algoritma sorting dalam struktur data.
2. Melatih kemampuan dalam mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
3. Membantu memahami proses pengurutan data melalui visualisasi langkah demi langkah.
4. Melatih kemampuan dalam membuat program GUI menggunakan Java Swing.
5. Memahami penerapan algoritma sorting pada data object menggunakan **ArrayList**.

BAB 2

PEMBAHASAN

2.1 Pengurutan menggunakan Insertion Sort

2.2.1 Method InsertionSort_2511531002()

```
1 package pekan7_2511531002;
2
3 public class InsertionSort_2511531002 {
4     public static void InsertionSort_2511531002(int[] arr_1002) {
5         int n_1002 = arr_1002.length;
6         for (int i_1002 = 1; i_1002 < n_1002; i_1002++) {
7             int key_1002 = arr_1002[i_1002];
8             int j_1002 = i_1002 - 1;
9             while (j_1002 ≥ 0 && arr_1002[j_1002] > key_1002) {
10                 arr_1002[j_1002 + 1] = arr_1002[j_1002];
11                 j_1002--;
12             }
13             arr_1002[j_1002 + 1] = key_1002;
14         }
15     }
16 }
```

Kode Program 2.1 – Blok Program Insertion Sort

Method ini digunakan untuk mengurutkan data array dengan metode *Insertion Sort*. Method ini menerima parameter berupa array integer yang akan diurutkan dari kecil ke besar.

Pada method ini dilakukan perulangan mulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut. Nilai pada indeks saat ini disimpan ke dalam variabel **key_1002**, kemudian dibandingkan dengan elemen-elemen sebelumnya menggunakan perulangan **while**.

Jika elemen sebelumnya lebih besar dari **key_1002**, maka elemen tersebut akan digeser satu posisi ke kanan. Proses ini dilakukan terus hingga ditemukan posisi yang sesuai. Setelah itu, nilai **key_1002** dimasukkan ke posisi tersebut sehingga array menjadi terurut.

2.1.2 Method main()

```
17 public static void main(String[] args) {  
18     int arr_1002[] = { 23, 78, 45, 8, 32, 56, 1 };  
19     int n_1002 = arr_1002.length;  
20     System.out.printf("array yang belum terurut:\n");  
21     for (int i_1002 = 0; i_1002 < n_1002; i_1002++)  
22         System.out.print(arr_1002[i_1002] + " ");  
23     System.out.println("");  
24     InsertionSort_2511531002(arr_1002);  
25     System.out.printf("array yang terurut:\n");  
26     for (int i_1002 = 0; i_1002 < n_1002; i_1002++)  
27         System.out.print(arr_1002[i_1002] + " ");  
28     System.out.println("");  
29 }  
30 }
```

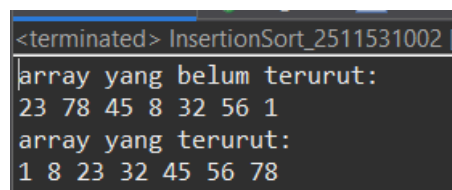
Kode Program 2.2 – Blok Program Main

Method ini digunakan untuk menjalankan program sekaligus menguji proses pengurutan metode *Insertion Sort*.

Pada method ini dibuat sebuah array integer **arr_1002** yang berisi beberapa data acak. Selanjutnya program menampilkan isi array sebelum diurutkan menggunakan perulangan **for**.

Setelah data awal ditampilkan, method **InsertionSort_2511531002(arr_1002)** dipanggil untuk melakukan proses pengurutan data. Setelah proses selesai, program kembali menampilkan isi array yang sudah terurut ke layar sehingga hasil sebelum dan sesudah pengurutan dapat dibandingkan.

Hasil output dari program adalah sebagai berikut:



```
<terminated> InsertionSort_2511531002 [  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

Gambar 2.1 – Hasil Program Pengurutan menggunakan Insetion Sort

2.2 Pengurutan menggunakan Selection Sort

2.2.1 Method SelectionSort_2511531002()

```
1 package pekan7_2511531002;
2
3 public class SelectionSort_2511531002 {
4     public static void SelectionSort_2511531002(int[] arr_1002) {
5         int n_1002 = arr_1002.length;
6         for (int i_1002 = 0; i_1002 < n_1002; i_1002++) {
7             int minIndex_1002 = i_1002;
8             for (int j_1002 = i_1002 + 1; j_1002 < n_1002; j_1002++) {
9                 if (arr_1002[j_1002] < arr_1002[minIndex_1002]) {
10                     minIndex_1002 = j_1002;
11                 }
12             }
13             int temp_1002 = arr_1002[i_1002];
14             arr_1002[i_1002] = arr_1002[minIndex_1002];
15             arr_1002[minIndex_1002] = temp_1002;
16         }
17     }
18 }
```

Kode Program 2.3 – Blok Program Selection Sort

Method ini digunakan untuk mengurutkan data array menggunakan metode *Selection Sort*. Method ini menerima parameter berupa array integer yang akan diurutkan dari kecil ke besar.

Pada method ini dilakukan perulangan untuk mencari nilai terkecil pada bagian array yang belum terurut. Indeks nilai terkecil tersebut disimpan ke dalam variabel **minIndex_1002**.

Selanjutnya dilakukan perbandingan menggunakan perulangan kedua untuk mencari elemen yang nilainya lebih kecil dari elemen pada **minIndex_1002**. Jika ditemukan nilai yang lebih kecil, maka indeks tersebut akan disimpan sebagai **minIndex_1002** terbaru.

Setelah proses pencarian selesai, data pada indeks awal akan ditukar dengan data pada **minIndex_1002** menggunakan variabel sementara **temp_1002**. Proses ini dilakukan berulang hingga seluruh data menjadi terurut.

2.2.2 Method main()

```
19 public static void main(String[] args) {  
20     int arr_1002[] = { 23, 78, 45, 8, 32, 56, 1 };  
21     int n_1002 = arr_1002.length;  
22     System.out.printf("array yang belum terurut:\n");  
23     for (int i_1002 = 0; i_1002 < n_1002; i_1002++)  
24         System.out.print(arr_1002[i_1002] + " ");  
25     System.out.println("");  
26     SelectionSort_2511531002(arr_1002);  
27     System.out.printf("array yang terurut:\n");  
28     for (int i_1002 = 0; i_1002 < n_1002; i_1002++)  
29         System.out.print(arr_1002[i_1002] + " ");  
30     System.out.println("");  
31 }  
32 }
```

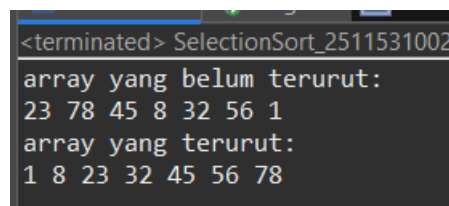
Kode Program 2.4 – Blok Program Main

Method ini digunakan untuk menjalankan program dan menguji proses pengurutan menggunakan metode *Selection Sort*.

Pada method ini dibuat sebuah array integer berisi beberapa data acak. Program kemudian menampilkan isi array sebelum diurutkan menggunakan perulangan **for**.

Selanjutnya method **SelectionSort_2511531002(arr_1002)** dipanggil untuk melakukan proses pengurutan data. Setelah proses selesai, program menampilkan kembali isi array yang sudah terurut ke layar.

Hasil output dari program adalah sebagai berikut:



```
<terminated> SelectionSort_2511531002  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

Gambar 2.2 – Hasil Program Pengurutan menggunakan Selection Sort

2.3 Pengurutan menggunakan Bubble Sort

2.3.1 Method BubbleSort_2511531002()

```
1 package pekan7_2511531002;
2
3 public class BubbleSort_2511531002 {
4     public static void BubbleSort_2511531002(int[] arr_1002) {
5         int n_1002 = arr_1002.length;
6         for (int i_1002 = 0; i_1002 < n_1002; i_1002++) {
7             for (int j_1002 = 0; j_1002 < n_1002 - i_1002 - 1; j_1002++) {
8                 if (arr_1002[j_1002] > arr_1002[j_1002 + 1]) {
9                     int temp_1002 = arr_1002[j_1002];
10                    arr_1002[j_1002] = arr_1002[j_1002 + 1];
11                    arr_1002[j_1002 + 1] = temp_1002;
12                }
13            }
14        }
15    }
```

Kode Program 2.5 – Blok Program Bubble Sort

Method ini digunakan untuk mengurutkan data array menggunakan metode *Bubble Sort*. Method ini menerima parameter berupa array integer yang akan diurutkan dari kecil ke besar.

Pada method ini dilakukan perulangan untuk membandingkan dua elemen yang bersebelahan. Jika elemen sebelah kiri lebih besar daripada elemen sebelah kanan, maka kedua elemen tersebut akan ditukar posisinya menggunakan variabel sementara **temp_1002**.

Proses perbandingan dilakukan berulang dari awal array hingga elemen terbesar berpindah ke posisi akhir. Setelah satu proses selesai, jumlah perbandingan akan berkurang karena data di bagian akhir sudah berada pada posisi yang benar. Proses ini terus dilakukan sampai seluruh data menjadi terurut.

2.3.2 Method main()

```
17 public static void main(String[] args) {  
18     int arr_1002[] = { 23, 78, 45, 8, 32, 56, 1 };  
19     int n_1002 = arr_1002.length;  
20     System.out.printf("array yang belum terurut:\n");  
21     for (int i_1002 = 0; i_1002 < n_1002; i_1002++)  
22         System.out.print(arr_1002[i_1002] + " ");  
23     System.out.println("");  
24     BubbleSort_2511531002(arr_1002);  
25     System.out.printf("array yang terurut:\n");  
26     for (int i_1002 = 0; i_1002 < n_1002; i_1002++)  
27         System.out.print(arr_1002[i_1002] + " ");  
28     System.out.println("");  
29 }  
30 }
```

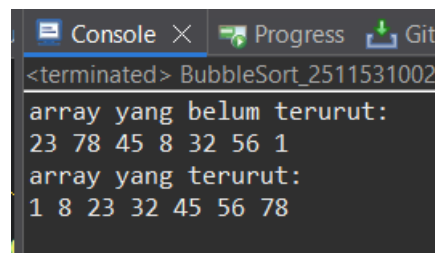
Kode Program 2.6 – Blok Program Main

Method ini digunakan untuk menjalankan program dan menguji proses pengurutan menggunakan metode *Bubble Sort*.

Pada method ini dibuat sebuah array integer berisi beberapa data acak. Program kemudian menampilkan isi array sebelum diurutkan menggunakan perulangan **for**.

Selanjutnya method **BubbleSort_2511531002(arr_1002)** dipanggil untuk melakukan proses pengurutan data. Setelah proses selesai, program menampilkan kembali isi array yang sudah terurut ke layar.

Hasil output dari program adalah sebagai berikut:



```
<terminated> BubbleSort_2511531002  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

Gambar 2. 3 – Hasil Program Pengurutan menggunakan Bubble Sort

2.4 Pengurutan menggunakan Insertion Sort menggunakan GUI

2.4.1 Deklarasi Class dan Variabel

```
1 package pekan7_2511531002;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 public class InsertionSortGUI_2511531002 extends JFrame {
22     private static final long serialVersionUID = 1L;
23     // private JPanel contentPane_1002;
24     private int[] array_1002;
25     private JLabel[] labelArray_1002;
26     private JButton stepButton_1002, resetButton_1002, setButton_1002;
27     private JTextField inputField_1002;
28     private JPanel panelArray_1002;
29     private JTextArea stepArea_1002;
30
31     private int i_1002 = 1, j_1002;
32     private boolean sorting_1002 = false;
33     private int stepCount_1002 = 1;
34 }
```

Kode Program 2.7 – Blok Program Deklarasi Class dan Variabel

Bagian program di atas merupakan deklarasi class **InsertionSortGUI_2511531002** yang merupakan turunan dari class **JFrame**. Pada bagian ini juga dilakukan deklarasi variabel yang digunakan dalam program, seperti array integer untuk menyimpan data, array **JLabel** untuk visualisasi array, tombol kontrol, field input, panel tampilan, serta variabel yang digunakan dalam proses *Insertion Sort* langkah demi langkah.

2.4.2 Constructor InsertionSortGUI_2511531002()

```
35 public InsertionSortGUI_2511531002() {
36     setTitle("Insertion Sort Langkah per Langkah");
37     setSize(750, 400);
38     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
39     setLocationRelativeTo(null);
40     setLayout(new BorderLayout());
}
```

Kode Program 2.8 – Blok Program Pengaturan Frame

Pada bagian ini dilakukan pengaturan tampilan utama GUI menggunakan class **JFrame**. Program menentukan judul window, ukuran tampilan, posisi window di tengah layar, serta layout utama menggunakan **BorderLayout**.

Selain itu, method **setDefaultCloseOperation** (**JFrame.EXIT_ON_CLOSE**) digunakan agar program otomatis berhenti ketika window ditutup.

```
// Panel Input
JPanel inputPanel_1002 = new JPanel(new FlowLayout());
inputField_1002 = new JTextField(30);
setButton_1002 = new JButton("Set Array");
inputPanel_1002.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
inputPanel_1002.add(inputField_1002);
inputPanel_1002.add(setButton_1002);
```

Kode Program 2.9 – Blok Program Pembuatan Panel Input

Pada bagian ini dibuat panel input menggunakan **JPanel** dengan **FlowLayout**. Panel ini digunakan untuk menampung komponen input seperti **JTextField**, tombol **Set Array**, dan **JLabel** petunjuk pengisian data.

JTextField digunakan untuk memasukkan data array yang dipisahkan dengan tanda koma, sedangkan tombol **Set Array** digunakan untuk memproses input array yang dimasukkan pengguna.

```
// Panel array visual
panelArray_1002 = new JPanel();
panelArray_1002.setLayout(new FlowLayout());
```

Kode Program 2.10 – Blok Program Pembuatan Panel Visual Array

Bagian ini digunakan untuk membuat **panelArray_1002** yang berfungsi menampilkan visualisasi array pada GUI. Panel menggunakan **FlowLayout** agar setiap elemen array yang ditampilkan dalam bentuk **JLabel** dapat tersusun secara horizontal.

```
// Panel kontrol
JPanel controlPanel_1002 = new JPanel();
stepButton_1002 = new JButton("Langkah Selanjutnya");
resetButton_1002 = new JButton("Reset");
stepButton_1002.setEnabled(false);
controlPanel_1002.add(stepButton_1002);
controlPanel_1002.add(resetButton_1002);
```

Kode Program 2.11 – Blok Program Pembuatan Panel Kontrol

Pada bagian ini dibuat panel kontrol yang berisi tombol **Langkah Selanjutnya** dan tombol **Reset**. Tombol **Langkah Selanjutnya** digunakan untuk menjalankan proses *Insertion Sort* satu langkah setiap kali ditekan, sedangkan tombol **Reset** digunakan untuk mengembalikan program ke kondisi awal.

Method **setEnabled(false)** digunakan agar tombol langkah tidak dapat ditekan sebelum array diinputkan.

```
// Area teks untuk log langkah-langkah
stepArea_1002 = new JTextArea(8, 60);
stepArea_1002.setEditable(false);
stepArea_1002.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_1002 = new JScrollPane(stepArea_1002);
```

Kode Program 2.12 – Blok Program Pembuatan Area Log Langkah
Sorting

Bagian ini digunakan untuk membuat **JTextArea** yang berfungsi menampilkan log proses sorting langkah demi langkah. **JTextArea** diatur menjadi tidak dapat diedit menggunakan **setEditable(false)**, kemudian font diubah menggunakan **Font** agar tampilan teks lebih rapi. Komponen ini kemudian dimasukkan ke dalam **JScrollPane** agar dapat dilakukan scrolling jika isi log terlalu panjang.

```
// Tambahkan panel ke frame
add(inputPanel_1002, BorderLayout.NORTH);
add(panelArray_1002, BorderLayout.CENTER);
add(controlPanel_1002, BorderLayout.SOUTH);
add(scrollPane_1002, BorderLayout.EAST);
```

Kode Program 2.13 – Blok Program Menambahkan Komponen ke Frame

Pada bagian ini seluruh panel dan komponen GUI ditambahkan ke dalam frame menggunakan method **add()**. Panel input ditempatkan di bagian atas (**BorderLayout.NORTH**), panel visual array di tengah (**BorderLayout.CENTER**), panel kontrol di bawah (**BorderLayout.SOUTH**), dan area log di bagian kanan (**BorderLayout.EAST**).

```
// Event Set Array, Langkah Selanjutnya, dan Reset
setButton_1002.addActionListener(e → setArrayFromInput_1002());
stepButton_1002.addActionListener(e → performStep_1002());
resetButton_1002.addActionListener(e → reset_1002());
}
```

Kode Program 2.14 – Blok Program Event Handling

Bagian ini digunakan untuk menghubungkan tombol dengan method tertentu menggunakan **ActionListener**. Tombol **Set Array** akan menjalankan method **setArrayFromInput_1002()**, tombol **Langkah Selanjutnya** menjalankan method **performStep_1002()**, dan tombol **Reset** menjalankan method **reset_1002()**. Dengan adanya event ini, program dapat merespon interaksi pengguna pada GUI.

2.4.3 Method setArrayFromInput_1002()

```
private void setArrayFromInput_1002() {
    String text_1002 = inputField_1002.getText().trim();
    if (text_1002.isEmpty()) return;
}
```

Kode Program 2.15 – Blok Program Mengambil Input Array

Pada bagian ini program mengambil input dari **inputField_1002** menggunakan method **getText()**, kemudian menghapus spasi di awal dan akhir teks menggunakan **trim()**. Jika input kosong, method akan langsung dihentikan menggunakan **return**.

```
String[] parts_1002 = text_1002.split(",");
array_1002 = new int[parts_1002.length];
try {
    for (int k_1002 = 0; k_1002 < parts_1002.length; k_1002++) {
        array_1002[k_1002] = Integer.parseInt(parts_1002[k_1002].trim());
    }
} catch (NumberFormatException e_1002) {
    JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
        + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
    return;
}
```

Kode Program 2.16 – Blok Program Memisahkan, Mengubah Input menjadi Integer, dan Penanganan Error Input

Bagian ini digunakan untuk memisahkan input berdasarkan tanda koma menggunakan method **split(",")**. Hasil pemisahan disimpan ke dalam array string **parts_1002**.

Selanjutnya dilakukan perulangan untuk mengubah setiap data string menjadi integer menggunakan **Integer.parseInt()**.

Lalu bagian selanjutnya digunakan blok **catch** untuk menangani kesalahan input. Jika pengguna memasukkan data selain angka, maka program akan menampilkan pesan error menggunakan **JOptionPane**.

```
i_1002 = 1;
stepCount_1002 = 1;
sorting_1002 = true;
stepButton_1002.setEnabled(true);
stepArea_1002.setText("");
```

Kode Program 2.17 – Blok Program Reset Variabel Sorting

Bagian ini digunakan untuk mengatur ulang variabel proses sorting serta mengaktifkan kembali tombol langkah sorting.

```
panelArray_1002.removeAll();
labelArray_1002 = new JLabel[array_1002.length];
for (int k_1002 = 0; k_1002 < array_1002.length; k_1002++) {
    labelArray_1002[k_1002] = new JLabel(String.valueOf(array_1002[k_1002]));
    labelArray_1002[k_1002].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_1002[k_1002].setBorder(BorderFactory.createLineBorder(Color.BLACK));
    labelArray_1002[k_1002].setPreferredSize(new Dimension(50, 50));
    labelArray_1002[k_1002].setHorizontalAlignment(SwingConstants.CENTER);
    panelArray_1002.add(labelArray_1002[k_1002]);
}
```

Kode Program 2.18 - Blok Program Pembuatan Visualisasi Array

Bagian ini digunakan untuk menghapus tampilan array sebelumnya dan membuat array **JLabel** baru. Perulangan setelahnya digunakan untuk membuat visualiasi setiap elemen array dalam bentuk **JLabel**, kemudian menampilkannya ke dalam **panelArray_1002**.

```
panelArray_1002.revalidate();
panelArray_1002.repaint();
}
```


Kode Program 2.19 – Blok Program Memperbarui Tampilan Panel

Bagian ini digunakan untuk memperbarui tampilan panel agar perubahan data dan visualisasi array dapat langsung terlihat pada GUI.

2.4.4 Method performStep_1002()

```
111● private void performStep_1002() {
112●     if (i_1002 < array_1002.length && sorting_1002) {
113         int key_1002 = array_1002[i_1002];
114         j_1002 = i_1002 - 1;
115
116         StringBuilder stepLog_1002 = new StringBuilder();
117         stepLog_1002.append("Langkah ").append(stepCount_1002).
118         append(": Memasukkan ").append(key_1002).append("\n");
119
120●         while (j_1002 ≥ 0 && array_1002[j_1002] > key_1002) {
121             array_1002[j_1002 + 1] = array_1002[j_1002];
122             j_1002--;
123         }
124         array_1002[j_1002 + 1] = key_1002;
125
126         updateLabels_1002();
127         stepLog_1002.append("Hasil: ").append(arrayToString_1002(array_1002)).append("\n\n");
128         stepArea_1002.append(stepLog_1002.toString());
129
130         i_1002++;
131         stepCount_1002++;
132
133●         if (i_1002 == array_1002.length) {
134             sorting_1002 = false;
135             stepButton_1002.setEnabled(false);
136             JOptionPane.showMessageDialog(this, "Sorting selesai!");
137         }
138     }
139 }
```

Kode Program 2.20 – Blok Program Menjalankan Proses Sort

Method ini digunakan untuk menjalankan proses *Insertion Sort* langkah demi langkah pada GUI. Method ini akan dijalankan setiap kali tombol **Langkah Selanjutnya** ditekan.

Pada awal method dilakukan pengecekan apakah proses sorting masih dapat dijalankan menggunakan kondisi **i_1002 < array_1002.length && sorting_1002**. Jika kondisi terpenuhi, maka program mengambil nilai array pada indeks saat ini dan menyimpannya ke dalam variabel **key_1002**. Selanjutnya variabel **j_1002** digunakan untuk membandingkan **key_1002** dengan elemen-elemen sebelumnya.

Program kemudian membuat log langkah sorting menggunakan **StringBuilder**. Log ini digunakan untuk menampilkan informasi langkah yang sedang diproses beserta hasil array setelah sorting dilakukan.

Proses *Insertion Sort* dilakukan menggunakan perulangan **while**. Jika elemen sebelumnya lebih besar dari **key_1002**, maka elemen tersebut

akan digeser satu posisi ke kanan hingga ditemukan posisi yang sesuai. Setelah posisi ditemukan, nilai **key_1002** dimasukkan ke posisi tersebut.

Setelah proses sorting selesai untuk satu langkah, method **updateLabels_1002()** dipanggil untuk memperbarui tampilan array pada GUI. Hasil array kemudian ditambahkan ke dalam **stepArea_1002** sebagai log proses sorting.

Di bagian akhir method, nilai **i_1002** dan **stepCount_1002** ditambah untuk melanjutkan proses ke langkah berikutnya. Jika seluruh data telah terurut, maka proses sorting dihentikan, tombol langkah dinonaktifkan, dan program menampilkan pesan bahwa proses sorting telah selesai.

2.4.5 Method updateLabels_1002()

```
private void updateLabels_1002() {  
    for (int k_1002 = 0; k_1002 < array_1002.length; k_1002++) {  
        labelArray_1002[k_1002].setText(String.valueOf(array_1002[k_1002]));  
    }  
}
```

Kode Program 2.21 – Blok Program Memperbarui Tampilan Array

Method ini digunakan untuk memperbarui tampilan array pada GUI setelah proses sorting dilakukan. Pada method ini dilakukan perulangan untuk mengakses string menggunakan **String.valueOf()** dan ditampilkan kembali pada **labelArray_1002** menggunakan method **setText()**. Dengan adanya method ini, perubahan data array setelah proses sorting dapat langsung terlihat pada tampilan GUI.

2.4.6 Method reset_1002()

```
147 private void reset_1002() {  
148     inputField_1002.setText("");  
149     panelArray_1002.removeAll();  
150     panelArray_1002.revalidate();  
151     panelArray_1002.repaint();  
152     stepArea_1002.setText("");  
153     stepButton_1002.setEnabled(false);  
154     sorting_1002 = false;  
155     i_1002 = 1;  
156     stepCount_1002 = 1;  
157 }
```

Kode Program 2.22 – Blok Program Reset

Method ini digunakan untuk mengembalikan tampilan dan variabel program ke kondisi awal. Pada method ini, isi **inputField_1002** dikosongkan menggunakan **setText("")**. Selanjutnya seluruh komponen visual array pada **panelArray_1002** dihapus menggunakan **removeAll()**, kemudian tampilan panel diperbarui kembali menggunakan **revalidate()** dan **repaint()**.

Area log langkah sorting pada **stepArea_1002** juga dikosongkan, sedangkan tombol **stepButton_1002** dinonaktifkan kembali agar proses sorting tidak dapat dijalankan sebelum array baru dimasukkan.

Selain itu, variabel proses sorting seperti **sorting_1002**, **i_1002**, dan **stepCount_1002** dikembalikan ke nilai awal sehingga program siap digunakan kembali dari awal.

2.4.7 Method updateLabels_1002()

```
private void updateLabels_1002() {  
    for (int k_1002 = 0; k_1002 < array_1002.length; k_1002++) {  
        labelArray_1002[k_1002].setText(String.valueOf(array_1002[k_1002]));  
    }  
}
```

Kode Program 2.23 – Blok Program Mengubah isi Array menjadi String

Method ini digunakan untuk mengubah isi array menjadi bentuk string. Pada method ini dibuat object **StringBuilder** bernama **sb_1002** yang digunakan untuk menyusun teks hasil array. Selanjutnya dilakukan perulangan untuk mengakses seluruh elemen array dan menambahkan nilainya ke dalam **StringBuilder**.

Jika elemen yang diproses bukan elemen terakhir, maka program akan menambahkan tanda koma dan spasi sebagai pemisah antar elemen array. Setelah seluruh elemen selesai diproses, method mengembalikan hasil string menggunakan **toString()**. Hasil ini digunakan untuk menampilkan isi array pada log langkah sorting di GUI.

2.4.8 Method main()

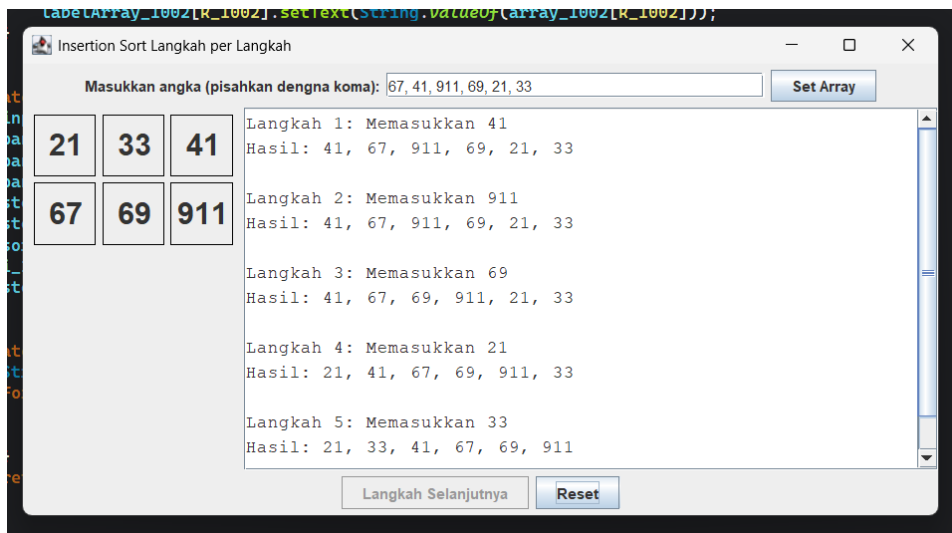
```
168 public static void main(String[] args) {  
169     SwingUtilities.invokeLater(() → {  
170         InsertionSortGUI_2511531002 gui_1002 = new InsertionSortGUI_2511531002();  
171         gui_1002.setVisible(true);  
172     });  
173 }  
174 }  
175 }
```

Kode Program 2.24 – Blok Program Main

Method ini digunakan sebagai titik awal untuk menjalankan program GUI *Insertion Sort*. Pada method ini digunakan **SwingUtilities.invokeLater()** untuk menjalankan GUI pada *Event Dispatch Thread* agar proses antarmuka berjalan dengan aman dan stabil.

Di dalam blok tersebut dibuat object **InsertionSortGUI_2511531002** dengan nama **gui_1002**, kemudian method **setVisible(true)** digunakan untuk menampilkan tampilan GUI ke layar sehingga program dapat digunakan oleh pengguna.

Hasil Output dari program adalah sebagai berikut:



Gambar 2. 4 – Hasil Program Insertion Sort dengan GUI

2.5 Latihan Sorting dengan GUI untuk mengurutkan Nama Mahasiswa

```
package pekan7_2511531002;

public class Mahasiswa_2511531002 {
    private String nama_1002;
    private String nim_1002;
    private String prodi_1002;

    public Mahasiswa_2511531002(String nama_1002, String
nim_1002, String prodi_1002){
        this.nama_1002 = nama_1002;
        this.nim_1002 = nim_1002;
        this.prodi_1002 = prodi_1002;
    }

    public String getNama_1002() { return nama_1002; }
    public String getNim_1002() { return nim_1002; }
    public String getProdi_1002() { return prodi_1002; }

    public void setNama_1002(String nama_1002) { this.nama_1002
= nama_1002; }
    public void setNim_1002(String nim_1002) { this.nim_1002 =
nim_1002; }
    public void setProdi_1002(String prodi_1002) {
this.prodi_1002 = prodi_1002; }

    @Override
    public String toString() { return nama_1002; }
}
```

Kode Program 2.25 – ADT Mahasiswa_2511531002

Class ini merupakan class yang digunakan untuk merepresentasikan data mahasiswa. Pada class ini terdapat atribut **nama_1002**, **nim_1002**, dan **prodi_1002** yang bertipe **String** untuk menyimpan nama mahasiswa, nomor induk mahasiswa, dan program studi.

Pada class ini juga terdapat constructor **Mahasiswa_2511531002(String nama_1002, String nim_1002, String prodi_1002)** yang digunakan untuk menginisialisasi data mahasiswa saat object dibuat. Nilai parameter yang diterima akan disimpan ke atribut class menggunakan keyword **this**.

Selain itu terdapat method getter seperti **getNama_1002()**, **getNim_1002()**, dan **getProdi_1002()** yang digunakan untuk mengambil nilai atribut. Terdapat juga method setter seperti **setNama_1002()**, **setNim_1002()**, dan **setProdi_1002()** yang digunakan untuk mengubah nilai atribut mahasiswa.

Pada class ini juga dilakukan override terhadap method **toString()** sehingga object mahasiswa akan menampilkan nilai **nama_1002** saat dipanggil atau ditampilkan pada komponen GUI.

```
package pekan7_2511531002;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridLayout;
import java.lang.reflect.Array;
import java.util.ArrayList;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class SortingGUI_2511531002 extends JFrame {

    private static final long serialVersionUID = 1L;

    private JTextField txtNama_1002, txtNim_1002,
txtProdi_1002;
    private JButton addButton_1002, stepButton_1002,
resetButton_1002;
    private JComboBox<String> comboSorting_1002;
    private JPanel panelArray_1002;
    private JTextArea areaLangkah_1002;
    private JLabel[] labelArray_1002;

    private ArrayList<Mahasiswa_2511531002> listMhs_1002;
    private ArrayList<Mahasiswa_2511531002> dataSorting_1002;

    private int i_1002;
    private boolean sortingSelesai_1002;

    public SortingGUI_2511531002() {
        setTitle("Visualisasi Sorting Mahasiswa");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
    }
}
```

```

listMhs_1002 = new ArrayList<>();

// Panel Input
JPanel panelInput_1002 = new JPanel(new GridLayout(5,
2, 5, 5));
panelInput_1002.setBorder(
    BorderFactory.createTitledBorder("Input
Data"));

panelInput_1002.add(new JLabel("Nama Mahasiswa"));
txtNama_1002 = new JTextField();
panelInput_1002.add(txtNama_1002);

panelInput_1002.add(new JLabel("NIM"));
txtNim_1002 = new JTextField();
panelInput_1002.add(txtNim_1002);

panelInput_1002.add(new JLabel("Program Studi"));
txtProdi_1002 = new JTextField();
panelInput_1002.add(txtProdi_1002);

panelInput_1002.add(new JLabel("Pilih Sotring"));
comboSorting_1002 = new JComboBox<>();

comboSorting_1002.addItem("Insertion Sort");
comboSorting_1002.addItem("Selection Sort");
comboSorting_1002.addItem("Bubble Sort");

panelInput_1002.add(comboSorting_1002);
addButton_1002 = new JButton("Tambah Data");
panelInput_1002.add(addButton_1002);

// Panel Visual Array
panelArray_1002 = new JPanel();
panelArray_1002.setLayout(new FlowLayout());
panelArray_1002.setBorder(
    BorderFactory.createTitledBorder("Visualisasi Data"));

// Panel Control
JPanel panelControl_1002 = new JPanel();
stepButton_1002 = new JButton("Langkah Selanjutnya");
resetButton_1002 = new JButton("Reset");
panelControl_1002.add(stepButton_1002);
panelControl_1002.add(resetButton_1002);

// Text Area
areaLangkah_1002 = new JTextArea();
areaLangkah_1002.setEditable(false);
areaLangkah_1002.setFont(new Font("Monospaced",
Font.PLAIN, 14));
JScrollPane scroll_1002 = new
JScrollPane(areaLangkah_1002);
scroll_1002.setPreferredSize(new Dimension(350,
500));

```

```

        // Add Component
        add(panelInput_1002, BorderLayout.NORTH);
        add(panelArray_1002, BorderLayout.CENTER);
        add(panelControl_1002, BorderLayout.SOUTH);
        add(scroll_1002, BorderLayout.EAST);

        // Button Event
        addButton_1002.addActionListener(e ->
tambahData_1002());
        stepButton_1002.addActionListener(e ->
prosesSorting_1002());
        resetButton_1002.addActionListener(e ->
reset_1002());
        stepButton_1002.setEnabled(false);
    }

    // Method Tambah Data
    private void tambahData_1002() {
        String nama_1002 = txtNama_1002.getText().trim();
        String nim_1002 = txtNim_1002.getText().trim();
        String prodi_1002 = txtProdi_1002.getText().trim();

        if (nama_1002.isEmpty() || nim_1002.isEmpty() ||
prodi_1002.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Data belum
lengkap!");
            return;
        }

        Mahasiswa_2511531002 mhs_1002 = new
Mahasiswa_2511531002(nama_1002, nim_1002, prodi_1002);
        listMhs_1002.add(mhs_1002);
        tampilkanVisual_1002(listMhs_1002);

        txtNama_1002.setText("");
        txtNim_1002.setText("");
        txtProdi_1002.setText("");
        stepButton_1002.setEnabled(true);

        i_1002 = 1;
        sortingSelesai_1002 = false;
        areaLangkah_1002.setText("");
    }

    // Visual Tlabel Array
    private void
tampilkanVisual_1002(ArrayList<Mahasiswa_2511531002> data_1002) {
        panelArray_1002.removeAll();
        labelArray_1002 = new JLabel[data_1002.size()];

        for (int k_1002 = 0; k_1002 < data_1002.size();
k_1002++) {
            labelArray_1002[k_1002] = new
JLabel(data_1002.get(k_1002).getNama_1002());
            labelArray_1002[k_1002].setFont(new
Font("Arial", Font.BOLD, 16));

```



```

        labelArray_1002[k_1002].setBorder(BorderFactory.createLineB
order(Color.BLACK));
        labelArray_1002[k_1002].setPreferredSize(new
Dimension(120, 50));

        labelArray_1002[k_1002].setHorizontalAlignment(SwingConstan
ts.CENTER);
        panelArray_1002.add(labelArray_1002[k_1002]);
    }
    panelArray_1002.revalidate();
    panelArray_1002.repaint();
}

// Proses Sorting Step
private void prosesSorting_1002() {
    if (listMhs_1002.size() <= 1) {
        JOptionPane.showMessageDialog(this, "Data
minimal 2!");
        return;
    }

    if (dataSorting_1002 == null || i_1002 == 1) {
        dataSorting_1002 = new ArrayList<>();
        for (Mahasiswa_2511531002 m : listMhs_1002) {
            dataSorting_1002.add(new
Mahasiswa_2511531002(
                                m.getNama_1002(),
m.getNama_1002(), m.getProdi_1002()));
        }

        String pilihan_1002 =
comboSorting_1002.getSelectedItem().toString();
        if (pilihan_1002.equals("Insetion Sort")) {
            insertionStep_1002();
        } else if (pilihan_1002.equals("Selection Sort")) {
            selectionStep_1002();
        } else {
            bubbleStep_1002();
        }

        tampilkanVisual_1002(dataSorting_1002);
    }

    // Insertion Sort Step
    private void insertionStep_1002() {
        if (i_1002 < dataSorting_1002.size()) {
            Mahasiswa_2511531002 key_1002 =
dataSorting_1002.get(i_1002);
            int j_1002 = i_1002 - 1;

            while (j_1002 >= 0 && dataSorting_1002.get(
j_1002).getNama_1002().compareToIgnoreCase(
key_1002.getNama_1002()) > 0) {

```

```

        dataSorting_1002.set(j_1002 + 1,
dataSorting_1002.get(j_1002));
        j_1002--;
    }

    dataSorting_1002.set(j_1002 + 1, key_1002);
    areaLangkah_1002.append(
        "Langkah " + i_1002 + " : " +
tampilNama_1002(
        dataSorting_1002) +
"\n");

    i_1002++;
    } else {
        selesai_1002();
    }
}

// Selection Sort Step
private void selectionStep_1002() {
    if (i_1002 < dataSorting_1002.size()) {
        int min_1002 = i_1002 - 1;
        for (int j_1002 = i_1002; j_1002 <
dataSorting_1002.size(); j_1002++) {
            if
(dataSorting_1002.get(j_1002).getNama_1002().
                compareToIgnoreCase(dataSorting_1002.get(min_1002).
getNama_1002())
< 0) {
                    min_1002 = j_1002;
                }
            }
            Mahasiswa_2511531002 temp_1002 =
dataSorting_1002.get(i_1002 - 1);
            dataSorting_1002.set(i_1002 - 1,
dataSorting_1002.get(min_1002));
            dataSorting_1002.set(min_1002, temp_1002);

            areaLangkah_1002.append("Pass " + i_1002 + " :
"
                + tampilNama_1002(dataSorting_1002)
+ "\n");

            i_1002++;
        } else {
            selesai_1002();
        }
    }

// Bubble Sort Step
private void bubbleStep_1002() {
    if (i_1002 < dataSorting_1002.size()) {
        for (int j_1002 = 0; j_1002 <
dataSorting_1002.size() - i_1002; j_1002++) {
            if
(dataSorting_1002.get(j_1002).getNama_1002().compareToIgnoreCase(

```

```

                                dataSorting_1002.get(j_1002
+ 1).getNama_1002()) > 0) {
                                Mahasiswa_2511531002 temp_1002 =
dataSorting_1002.get(j_1002);
                                dataSorting_1002.set(j_1002,
dataSorting_1002.get(j_1002 + 1));
                                dataSorting_1002.set(j_1002 + 1,
temp_1002);
                                }
                                }
                                areaLangkah_1002.append("Pass " + i_1002 + " :
" + tampilNama_1002(
                                dataSorting_1002) + "\n");

                                i_1002++;
                                } else {
                                selesai_1002();
                                }
                                }

// String Array
private String
tampilNama_1002(ArrayList<Mahasiswa_2511531002> data_1002) {
    String hasil_1002 = "[";

    for (int i_1002 = 0; i_1002 < data_1002.size();
i_1002++) {
        hasil_1002 +=
data_1002.get(i_1002).getNama_1002();
        if (i_1002 != data_1002.size() - 1) {
            hasil_1002 += ", ";
        }
        hasil_1002 += "];"
        return hasil_1002;
    }

// Sorting Selesai
private void selesai_1002() {
    if (!sortingSelesai_1002) {
        sortingSelesai_1002 = true;
        stepButton_1002.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
selesai!");
    }
}

// Reset
private void reset_1002() {
    listMhs_1002.clear();
    dataSorting_1002 = null;
    panelArray_1002.removeAll();
    panelArray_1002.revalidate();
    panelArray_1002.repaint();

    areaLangkah_1002.setText("");
    txtNama_1002.setText("");

```

```

        txtNim_1002.setText("");
        txtProdi_1002.setText("");

        stepButton_1002.setEnabled(false);
        i_1002 = 1;
        sortingSelesai_1002 = false;
    }

    // Main
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new SortingGUI_2511531002().setVisible(true);
        });
    }
}

```

Kode Program 2.16 – Tugas RumahSakit_2511531002

Class ini merupakan class yang digunakan untuk menampilkan visualisasi proses sorting data mahasiswa menggunakan GUI (*Graphical User Interface*). Program dibuat menggunakan Java Swing dan mendukung tiga metode sorting yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.

Pada bagian awal class dilakukan deklarasi berbagai komponen GUI seperti **JTextField**, **JButton**, **JComboBox**, **JPanel**, **JTextArea**, dan **JLabel**. Komponen tersebut digunakan untuk membuat tampilan input data mahasiswa, visualisasi array, area log langkah sorting, serta tombol kontrol program. Selain itu terdapat **ArrayList<Mahasiswa_2511531002>** yang digunakan untuk menyimpan data mahasiswa asli dan data yang sedang diproses sorting.

Constructor **SortingGUI_2511531002()** digunakan untuk mengatur tampilan awal GUI program. Pada constructor ini ditentukan ukuran frame, layout, serta posisi tampilan window. Program kemudian membuat panel input yang berisi field nama, NIM, program studi, dan pilihan metode sorting menggunakan **JComboBox**. Selain itu dibuat panel visualisasi data, panel kontrol, serta **JTextArea** untuk menampilkan log langkah sorting. Di bagian akhir constructor juga dipasang **ActionListener** pada tombol agar program dapat merespon interaksi pengguna.

Method **tambahData_1002()** digunakan untuk mengambil input data mahasiswa dari pengguna. Data nama, NIM, dan program studi diambil dari **JTextField**, kemudian dicek apakah masih ada data yang kosong. Jika seluruh data

lengkap, program akan membuat object **Mahasiswa_2511531002** dan menambahkannya ke dalam **listMhs_1002**. Setelah itu tampilan visual data diperbarui menggunakan method **tampilkanVisual_1002()**.

Method **tampilkanVisual_1002(ArrayList<Mahasiswa_2511531002> data_1002)** digunakan untuk menampilkan data mahasiswa secara visual pada GUI. Pada method ini setiap data mahasiswa ditampilkan dalam bentuk **JLabel** dengan pengaturan ukuran, border, font, dan posisi teks agar tampilan lebih rapi dan mudah dibaca.

Method **prosesSorting_1002()** digunakan untuk menjalankan proses sorting berdasarkan metode yang dipilih pengguna. Program terlebih dahulu mengecek jumlah data mahasiswa, kemudian membuat salinan data ke dalam **dataSorting_1002** agar data asli tidak berubah. Setelah itu program mengambil pilihan sorting dari **comboSorting_1002** dan menjalankan method sorting yang sesuai, yaitu **insertionStep_1002()**, **selectionStep_1002()**, atau **bubbleStep_1002()**.

Method **insertionStep_1002()** digunakan untuk menjalankan proses *Insertion Sort* langkah demi langkah. Program menyimpan data mahasiswa saat ini sebagai **key_1002**, kemudian membandingkannya dengan data sebelumnya berdasarkan nama mahasiswa menggunakan **compareToIgnoreCase()**. Jika urutan belum sesuai, maka data akan digeser hingga ditemukan posisi yang tepat.

Method **selectionStep_1002()** digunakan untuk menjalankan proses *Selection Sort* langkah demi langkah. Pada method ini program mencari nama mahasiswa terkecil menggunakan perulangan, kemudian menukarkan posisinya dengan data pada indeks awal menggunakan variabel sementara.

Method **bubbleStep_1002()** digunakan untuk menjalankan proses *Bubble Sort* langkah demi langkah. Program membandingkan dua data mahasiswa yang bersebelahan berdasarkan nama mahasiswa. Jika urutannya salah, maka kedua data akan ditukar posisinya hingga seluruh data menjadi terurut.

Method **tampilNama_1002(ArrayList<Mahasiswa_2511531002> data_1002)** digunakan untuk mengubah isi data mahasiswa menjadi bentuk string.

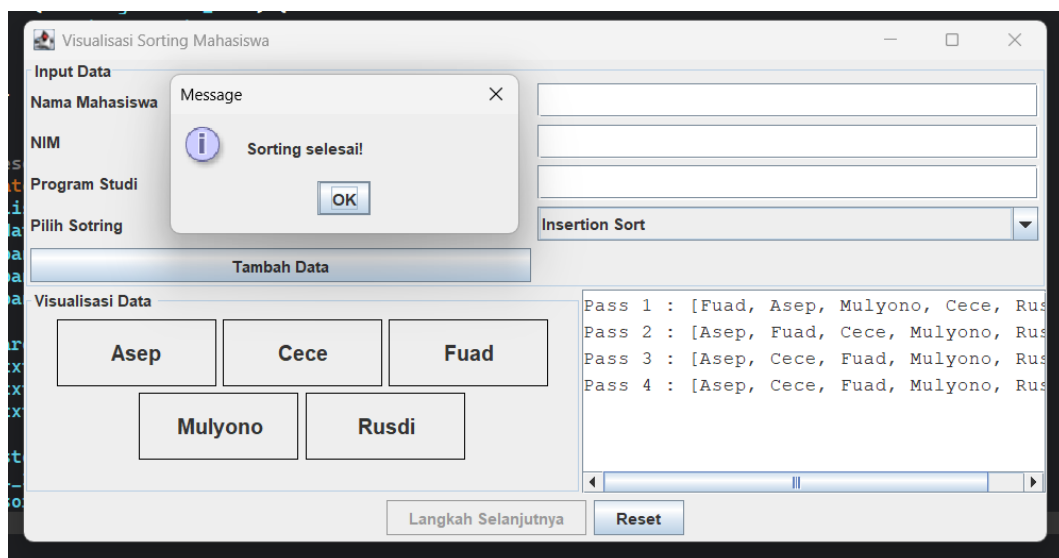
Method ini menggabungkan nama seluruh mahasiswa ke dalam satu string yang digunakan untuk menampilkan log proses sorting pada **areaLangkah_1002**.

Method **selesai_1002()** digunakan untuk menandai bahwa proses sorting telah selesai. Pada method ini tombol langkah sorting dinonaktifkan dan program menampilkan pesan menggunakan **JOptionPane**.

Method **reset_1002()** digunakan untuk mengembalikan program ke kondisi awal. Pada method ini seluruh data mahasiswa dihapus, panel visual dibersihkan, area log dikosongkan, serta field input dan variabel proses sorting dikembalikan ke nilai awal.

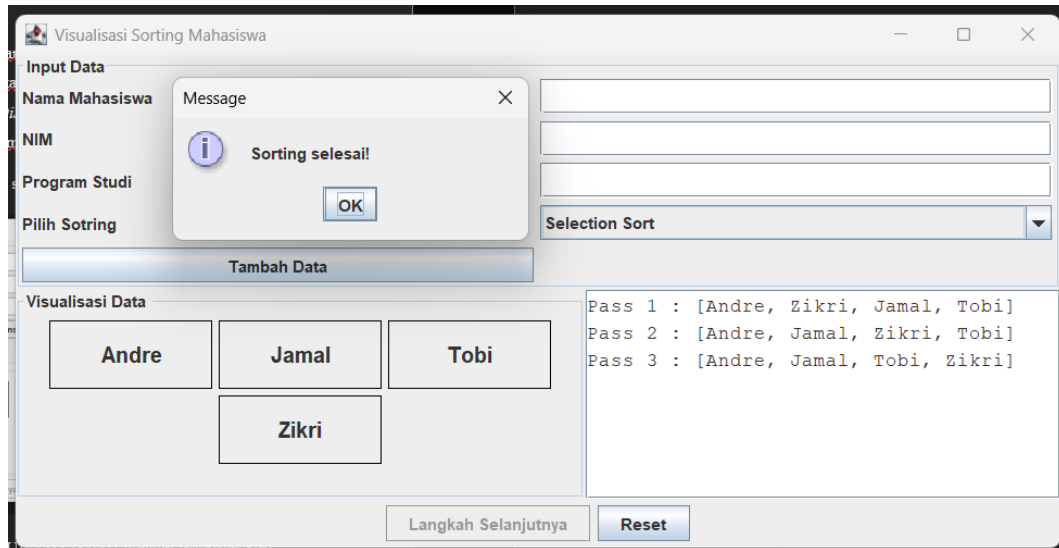
Terakhir, method **main()** digunakan sebagai titik awal untuk menjalankan program GUI. Pada method ini digunakan **SwingUtilities.invokeLater()** untuk menjalankan GUI pada *Event Dispatch Thread*, kemudian object **SortingGUI_2511531002** dibuat dan ditampilkan menggunakan **setVisible(true)**.

Hasil output program akan terlihat seperti berikut:



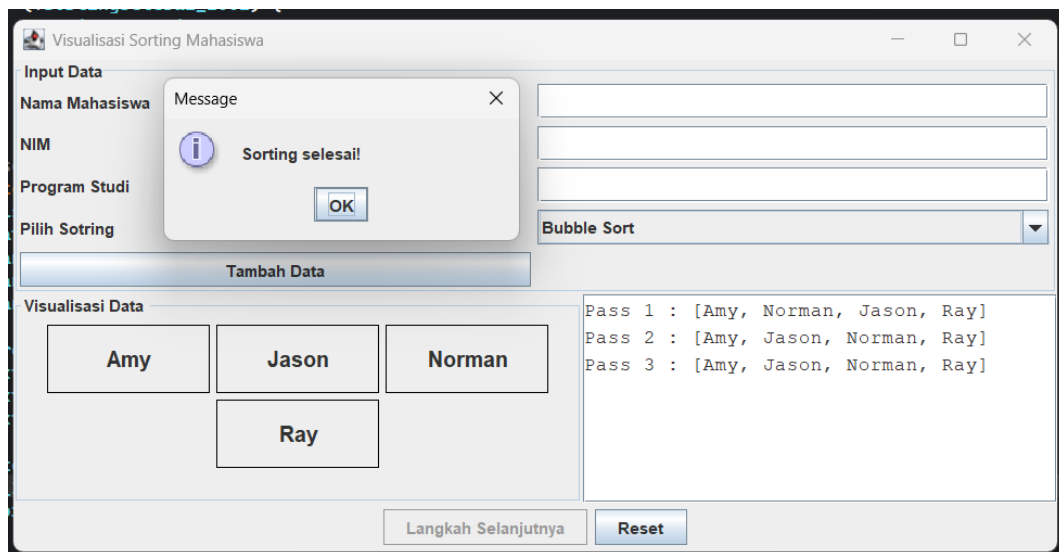
Gambar 2. 5 – Hasil Program SortingGUI_2511531002 dengan Insertion Sort

Urutan Awal: Asep, Fuad, Mulyono, Cece, Rusdi



Gambar 2. 6 – Hasil Program SortingGUI_2511531002 dengan Selection Sort

Urutan Awal: Zikri, Andre, Jamal, Tobi



Gambar 2. 7 – Hasil Program SortingGUI_2511531002 dengan Bubble Sort

Urutan Awal: Norman, Amy, Ray, Jason

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih terstruktur dan mudah diproses. Pada praktikum ini digunakan tiga metode sorting yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* yang masing-masing memiliki cara kerja berbeda dalam melakukan pengurutan data.

Implementasi sorting dilakukan menggunakan bahasa pemrograman Java baik dalam bentuk program console maupun GUI. Pada program GUI, proses sorting dapat ditampilkan secara visual dan langkah demi langkah sehingga mempermudah pemahaman terhadap alur perpindahan data selama proses pengurutan berlangsung.

Selain itu, praktikum ini juga menunjukkan bahwa algoritma sorting tidak hanya dapat digunakan pada data integer, tetapi juga dapat diterapkan pada object seperti data mahasiswa menggunakan **ArrayList** dan proses perbandingan berdasarkan nama mahasiswa.

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 7 – Sorting